COMPUTER SCIENCE FUNDAMENTALS

A University-Level Study Guide

Concepts • Worked Examples • Practice • Answer Key

MATHGENIUS

Solve Smarter. Learn Better.

For university, polytechnic and college learners

About This Guide

This guide introduces core ideas that appear across Computer Science programmes. It is designed for beginners and early university-level learners. It combines plain-language explanations with examples that connect theory to practical computing.

How to study: Read one chapter at a time, explain concepts in your own words, trace examples by hand, and attempt the practice questions before checking the answer key.

Scope: This is a foundation guide, not a complete substitute for lectures, laboratory work, prescribed textbooks or your institution's syllabus.

Contents

1. What Computer Science Is

2. Data and Number Systems

3. Computer Hardware and Architecture

4. Software and Operating Systems

5. Algorithms and Problem Solving

6. Programming Fundamentals

7. Data Structures

8. Databases and SQL

9. Networks and the Internet

10. Cybersecurity

11. Software Development and Testing

12. Web and Cloud Technologies

13. AI and Data Science

14. Revision and Key Terms

15. Practice Questions and Answers

1. What Computer Science Is

Computer Science studies computation: how problems are represented, how instructions are designed to solve them, and how computer systems execute those instructions.

Main areas

Programming involves writing instructions. Algorithms describe methods for solving problems. Computer systems examine hardware and software. Data management covers storing and retrieving information. Networks connect devices. Other areas include cybersecurity, artificial intelligence, human-computer interaction and theoretical computing.

Computer Science and IT

Computer Science focuses strongly on computation, algorithms, programming and the principles behind computing systems. Information Technology often focuses on selecting, deploying, maintaining and supporting technology in organisations. The fields overlap.

Input–process–output model

A computer receives data (input), performs operations (processing), and presents a result (output). Storage keeps data and instructions for later use. A calculator, for example, receives two numbers, adds them and displays the sum.

Example: student-result system

Input: student scores and ID.

Processing: validate scores, calculate total and average, then apply grading rules.

Output: a result report. Storage: save it securely for authorised retrieval.

2. Data and Number Systems

Computers represent digital content using bits. Data representation helps explain memory sizes, file formats, images, text and machine instructions.

Data, information and knowledge

Data is raw facts, such as 72, 81 and 65. Information is processed data, such as the average score. Knowledge involves interpreting information and applying it to decisions.

Bits and bytes

A bit is a binary digit, 0 or 1. Eight bits make one byte. Storage is commonly described in KB, MB, GB and TB. Decimal units often use 1 GB = 1,000 MB; binary units use 1 GiB = 1,024 MiB.

Binary, decimal and hexadecimal

Decimal uses digits 0–9 and powers of 10. Binary uses 0 and 1 and powers of 2. Hexadecimal uses 0–9 and A–F and powers of 16. Hexadecimal is common in memory addresses, colour codes and debugging.

Example: binary 1011 to decimal

Place values from right to left are 1, 2, 4 and 8.

(1 × 8) + (0 × 4) + (1 × 2) + (1 × 1) = **11**.

Example: decimal 13 to binary

Repeated division by 2 gives remainders 1, 0, 1, 1 (read from bottom to top).

Therefore 13 in decimal is **1101** in binary.

Text, images and sound

Text uses character standards such as Unicode. Images are represented by pixels and colour values. Digital audio stores samples of a sound wave. Lossless compression preserves original data; lossy compression discards some information to reduce file size.

3. Computer Hardware and Architecture

Hardware means the physical components of a computer. Architecture describes how components are organised and how they work together.

Main components

The CPU executes instructions. RAM temporarily holds data and programmes in use. Storage keeps files when power is off. The motherboard connects components. Input devices include keyboards and scanners; output devices include monitors and printers.

The CPU

The control unit coordinates instruction execution. The arithmetic logic unit (ALU) performs arithmetic and logical operations. Registers are small, fast storage locations inside the CPU.

Fetch–decode–execute cycle

The CPU fetches an instruction from memory, decodes what it means, executes the operation and updates system state. Modern processors use caches and pipelining to improve performance.

Primary and secondary storage

RAM is volatile: its contents are normally lost when power is removed. ROM and flash memory are non-volatile. SSDs use flash memory; HDDs store data magnetically on rotating platters. Cache is small, high-speed memory.

Choosing storage

An SSD can improve boot and application loading times.

An HDD may be useful for large archives where cost per gigabyte matters.

A computer can use an SSD for its operating system and an HDD for bulk storage.

4. Software and Operating Systems

Software is a set of instructions and related data that tells a computer what to do.

Types of software

System software manages the computer and provides a platform for applications. Examples include operating systems and device drivers. Application software helps users perform tasks such as word processing or browsing. Utility software supports maintenance, backup and security.

Operating system functions

An operating system manages processes, memory, files, devices, users and security. It provides an interface through which users and applications interact with hardware. Examples include Windows, Linux, macOS, Android and iOS.

Files and file systems

A file is a named collection of data. A directory (folder) organises files. A file system defines how data is named, stored and retrieved. Extensions such as .pdf, .jpg, .html and .py often indicate a format but do not guarantee file contents.

Processes and multitasking

A programme is instructions stored on a device; a process is a programme being executed. The OS schedules processes so multiple tasks can make progress. Threads are units of execution within a process.

5. Algorithms and Problem Solving

An algorithm is a finite, ordered set of unambiguous steps for solving a problem. A good algorithm should be correct, clear and sufficiently efficient.

Designing an algorithm

Understand the problem, identify inputs and outputs, break it into steps, choose data structures, write pseudocode or draw a flowchart, then test normal, boundary and unusual inputs.

Pseudocode and flowcharts

Pseudocode expresses an algorithm in structured plain language. Flowcharts use an oval for start/end, parallelogram for input/output, rectangle for processing and diamond for a decision.

Example: find the larger of two numbers

START; INPUT A, B; IF A > B DISPLAY A; ELSE IF B > A DISPLAY B; ELSE DISPLAY “Equal”; STOP.

Searching and sorting

Linear search checks items one by one and works on unsorted data. Binary search repeatedly halves the search range but requires sorted data. Sorting arranges items, for example in ascending order. Selection sort repeatedly selects the smallest remaining item; merge sort divides, sorts and combines.

Big-O notation

Big-O describes how resource use grows with input size n. O(1) is constant, O(log n) logarithmic, O(n) linear, O(n log n) common in efficient sorting, and $O(n^2)$ often appears in simple nested loops. It describes growth, not exact running time.

Example: complexity

A loop visits each of n items once and does constant work per item.

The work grows in proportion to n, so the time complexity is **O(n)**.

6. Programming Fundamentals

Programming expresses algorithms in a language that a computer can execute.

Variables and data types

A variable is a named reference to a value. Common types include integers, floating-point numbers, strings and booleans (true/false).

Operators and expressions

Arithmetic operators include +, -, *, / and remainder. Comparison operators test equality and ordering. Logical operators include AND, OR and NOT. Parentheses help make precedence clear.

Selection and iteration

Selection chooses a path based on a condition, using if/else. Iteration repeats instructions using loops such as for and while. A loop needs a stopping condition.

Functions and modularity

A function is a reusable block of code. It may receive parameters and return a result. Functions make programmes easier to understand, test and maintain.

Example: rectangle area in Python

```
def area(length, width):
return length * width
print(area(5, 3))
```

The function returns **15**.

Errors and debugging

A syntax error breaks language grammar. A runtime error occurs during execution, such as division by zero. A logic error runs but produces an incorrect result. Debugging means finding and fixing defects.

7. Data Structures

A data structure organises data so it can be accessed and modified efficiently.

Arrays and lists

An array stores a sequence of elements, often of the same type, and supports access by index. Lists are flexible sequences in many languages. Indexing commonly starts at 0.

Stacks and queues

A stack follows Last In, First Out (LIFO): the last item added is removed first. Push adds; pop removes. A queue follows First In, First Out (FIFO): the earliest item is removed first. Enqueue adds; dequeue removes.

Linked lists, trees and graphs

A linked list stores nodes connected by references. A tree organises nodes hierarchically with a root and children. A graph contains vertices and edges and can represent roads or social connections.

Hash tables

A hash function maps a key to a storage location. Hash tables can support fast average-case lookup. A collision occurs when different keys map to the same location; methods include chaining and open addressing.

Choosing a structure

Undo history: stack, because the most recent action is undone first.

Print jobs: queue, because jobs are handled in arrival order.

Family hierarchy: tree, to represent parent–child relationships.

8. Databases and SQL

A database is an organised collection of related data. A database management system (DBMS) defines, stores, queries, updates and protects data.

Relational databases

A relational database organises data into tables. A row is a record; a column is an attribute. A primary key uniquely identifies a row. A foreign key refers to a key in another table.

SQL basics

SQL retrieves and modifies relational data. SELECT retrieves, INSERT adds, UPDATE changes and DELETE removes records. WHERE filters rows.

Example: SQL query

```
SELECT name, score
FROM Students
WHERE score >= 50;
```

This returns names and scores for students scoring at least 50.

Database design and normalisation

Good design reduces unnecessary duplication and improves consistency. Normalisation organises tables and relationships to reduce update anomalies. The appropriate design depends on system requirements.

Transactions and ACID

Atomicity means all or nothing. Consistency preserves valid rules. Isolation prevents concurrent transactions from interfering improperly. Durability means committed changes persist.

9. Networks and the Internet

A network connects devices so they can communicate and share resources.

Network types and devices

A LAN covers a limited area such as an office or campus. A WAN spans a larger area. A router forwards packets between networks. A switch connects devices within a local network. A modem connects a network to an internet service over the provider's access technology.

IP and MAC addresses

An IP address identifies a network interface for communication across IP networks. IPv4 uses 32 bits; IPv6 uses 128 bits. A MAC address is a link-layer identifier used on local networks.

Protocols and the web

A protocol is a set of communication rules. TCP supports reliable, ordered delivery. IP handles addressing and routing. DNS translates domain names into IP addresses. HTTP transfers web resources; HTTPS protects HTTP communication using TLS.

Client–server model

A client requests a service or resource; a server provides it. A browser requesting a page is a client, while the web server responds. Some systems use peer-to-peer communication.

What happens when you open a website?

You enter a domain name.

DNS helps find the server's IP address.

The browser sends an HTTP or HTTPS request.

The server returns resources and the browser renders the page.

10. Cybersecurity

Cybersecurity protects systems, networks and data from unauthorised access, damage and disruption.

CIA triad

Confidentiality means information is available only to authorised people. Integrity means information remains accurate and is not improperly changed. Availability means systems and information are accessible when needed.

Common threats

Malware includes viruses, worms, trojans and ransomware. Phishing tricks people into revealing information or opening malicious links. Social engineering manipulates people. Weak or reused passwords increase account risk.

Practical protection

Use long, unique passwords and a password manager. Enable multi-factor authentication. Keep software updated. Back up important files. Check links and sender details. Limit permissions.

Encryption, hashing and authentication

Encryption transforms readable data into ciphertext using a key so authorised parties can recover it. Hashing maps data to a fixed-length digest and helps verify integrity. Authentication checks identity; authorisation determines permitted actions.

Important: Never store passwords as plain text. Real systems should use a dedicated password-hashing scheme with salts and appropriate work factors.

11. Software Development and Testing

Software engineering applies disciplined methods to building and maintaining reliable software.

Software development life cycle

A typical life cycle includes requirements analysis, design, implementation, testing, deployment and maintenance. Real projects may repeat these stages.

Development approaches

Waterfall organises work into sequential phases and can suit stable requirements. Agile approaches deliver smaller increments and use feedback to adapt. The best approach depends on risk, team, users and constraints.

Testing levels

Unit testing checks small components. Integration testing checks whether components work together. System testing checks the complete system. Acceptance testing checks user needs. Regression testing checks that changes have not broken existing behaviour.

Quality attributes

Useful software should be correct, secure, usable, reliable, maintainable and efficient. Clear requirements, documentation and version control support quality.

Example: testing a login form

Correct registered email and password: grant access.

Correct email and wrong password: deny access safely.

Empty fields: request missing information.

Repeated failed attempts: apply suitable rate limiting or other protective controls.

12. Web and Cloud Technologies

The web is a system of linked resources accessed over the internet.

HTML, CSS and JavaScript

HTML gives a page structure and meaning. CSS controls presentation and layout. JavaScript adds behaviour and interactivity in the browser.

Front end and back end

The front end is the part users interact with, often in a browser. The back end handles server-side logic, data access and authentication. An API defines how software components communicate.

Static and dynamic websites

A static site can serve prebuilt files directly and may not need a server-side application. A dynamic site generates or personalises content using application logic and often a database. Advanced features may need a backend or external service.

Cloud computing

Cloud computing provides computing resources over a network. IaaS offers infrastructure, PaaS provides a managed platform, and SaaS delivers ready-to-use software. Cloud services require attention to cost, security, availability and data location.

13. AI and Data Science

Artificial Intelligence (AI) concerns systems that perform tasks associated with intelligent behaviour. Machine learning is a subset of AI in which models learn patterns from data.

Data science workflow

Define a question, collect data, clean and explore it, select a method, evaluate results and communicate findings. Poor-quality or biased data can lead to misleading conclusions.

Machine learning types

Supervised learning uses labelled examples. Unsupervised learning finds structure in unlabelled data. Reinforcement learning trains an agent through actions and feedback from an environment.

Training, validation and testing

Training data fits the model; validation data helps tune choices; test data estimates performance on unseen examples. Overfitting occurs when a model learns training details too closely and performs poorly on new data.

Responsible AI

AI can produce inaccurate or biased outputs. Responsible development considers privacy, fairness, transparency, safety and human oversight. Results should be checked rather than accepted blindly.

14. Revision and Key Terms

Use this chapter to organise revision and build confidence with Computer Science vocabulary.

Study routine

Study in focused sessions. After reading a concept, close the guide and explain it from memory. Write a small programme or draw a diagram where appropriate. Attempt questions without looking at the answer key, then review mistakes.

Key terms

Algorithm: finite steps for solving a problem. Bit: binary digit. Byte: eight bits. CPU: central processing unit. RAM: volatile working memory. OS: operating system. Variable: named reference to a value. Function: reusable code unit. Data structure: organised way to store data. DBMS: database management system. Protocol: communication rules. API: interface for software communication.

Quick reminders

Binary place values are powers of 2. One byte contains 8 bits. Average = sum of values ÷ number of values. Big-O describes growth in resource use as input size increases. These reminders are not a complete list for every course.

15. Practice Questions

Attempt these questions before checking the answer key. For programming questions, write your reasoning or test your solution in a safe development environment.

1. **What does Computer Science study?**
2. **Differentiate data and information.**
3. **Convert binary 1001 to decimal.**
4. **How many bits are in a byte?**
5. **Name three CPU components.**
6. **Why is RAM volatile?**
7. **State two operating-system responsibilities.**
8. **Define an algorithm.**
9. **What does binary search require?**
10. **What does O(n) mean?**
11. **Syntax error versus logic error?**
12. **Which structure follows LIFO?**
13. **Which structure supports first-come, first-served processing?**
14. **What is a primary key?**
15. **What does SQL SELECT do?**
16. **What is DNS used for?**
17. **Name the CIA triad.**
18. **Define phishing.**
19. **HTML versus CSS?**
20. **What is overfitting?**

Answer Key and Explanations

1. What does Computer Science study?
Computation, algorithms, programming and computer systems.

2. Differentiate data and information.
Data is raw facts; information is data processed into a useful form.

3. Convert binary 1001 to decimal.
9, because 8 + 1 = 9.

4. How many bits are in a byte?
8 bits.

5. Name three CPU components.
Control unit, ALU and registers.

6. Why is RAM volatile?
Its contents are normally lost when power is removed.

7. State two operating-system responsibilities.
Examples: managing memory, processes, files, devices, users or security.

8. Define an algorithm.
A finite, ordered set of clear steps for solving a problem.

9. What does binary search require?
Sorted data.

10. What does O(n) mean?
Work grows approximately in proportion to input size n.

11. Syntax error versus logic error?
Syntax violates language rules; logic runs but gives an incorrect result.

12. Which structure follows LIFO?
A stack.

13. Which structure supports first-come, first-served processing?
A queue.

14. What is a primary key?
A field or combination of fields that uniquely identifies a table record.

15. What does SQL SELECT do?
Retrieves data from a database.

16. What is DNS used for?
Helping translate domain names into IP addresses.

17. Name the CIA triad.
Confidentiality, Integrity and Availability.

18. Define phishing.
A deceptive attempt to trick someone into revealing information or taking an unsafe action.

19. HTML versus CSS?
HTML structures content; CSS controls presentation and layout.

20. What is overfitting?
When a model fits training data too closely and performs poorly on new data.

End of guide. Keep learning, practise regularly, and consult your course outline for topics specific to your institution and programme.

MATHGENIUS • Solve Smarter. Learn Better.